# Bird Sound Classification using Deep Learning

Fine-tuning Resnets and Transformers on BirdCLEF 2021

Karine Alhakim, Leo Assini, Gašper Azinovič, Serge Bressloff, Veronica Cornwell, Cenzo Poirier De Carvalho

github.com/SpehKing/bird-sound-classification

## 1 Problem Formulation

BirdCLEF is a Kaggle competition (https://www.kaggle.com/c/birdclef-2021) organized every year since 2021 by the Cornell Lab of Ornithology. This competition addresses how to detect which bird species are present in audio recordings using machine learning techniques. We are given a raw audio clip (8-30 seconds), and a model must predict which of the 397 species from America (North and South) produced the sound. This is a multi-class classification problem with significant real-world applications in ecological monitoring, biodiversity research and citizen science initiatives.

### 1.1 Dataset

In order to be able to reach reasonable accuracies, not deal with class imbalance, and be able to train our model on lighter data, we use a curated subset of 10 species from the total dataset, composed of the 10 most populated classes:

- **Total recordings:** $5,000$ (500 per species)
- **Audio format:** '.ogg' files, 8-30 seconds per recording
- **Data split:** Stratified into train ($4,000$ recordings / 400 per species), validation (500 / 50 per species), and test (500 / 50 per species)

### 1.2 Data Preprocessing

All audio is resampled to 16 kHz and chunked into non-overlapping 10-second segments, zero-padding if necessary. This produces approximately 24,000 training chunks, 3,000 validation chunks, and 3,000 test chunks.

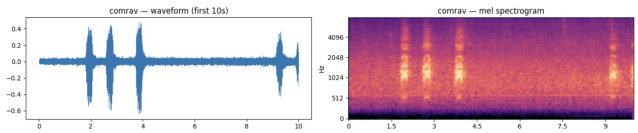



Figure 1: Example audio of a common raven in waveform and spectrogram formats

### 1.3 Evaluation Metrics

We report two accuracy metrics:

- **Chunk-level accuracy:** Fraction of 10-second chunks classified correctly (one prediction per chunk).
- **Recording-level accuracy:** Fraction of full recordings classified correctly after averaging chunk logits and taking argmax.

Recording-level accuracy is the primary metric because it reflects how the model would be deployed in practice.

## 2 Model Design Choices

### 2.1 AlexNet

To establish a lighter, foundational baseline against our deeper architectures (ResNet) and modern transformers (AST), we evaluated **AlexNet**. Given its historical significance and relatively shallow convolutional structure, AlexNet provides a useful data point on the trade-off between computational efficiency and feature extraction capability when applying standard image-based architectures to audio spectrograms.

#### 2.1.1 Training Strategy

Here, we employ a two-phase transfer-learning strategy:

- **Head-only:** Freeze the backbone and train only the new classifier head. This lets the randomly-initialised head catch up without corrupting the pretrained features with large gradients.
- **Full fine-tuning:** Unfreeze all layers and train end-to-end, using a $10\times$ smaller LR on the backbone than on the head so the ImageNet features adapt gently. A cosine annealing schedule decays both LRs over this phase.

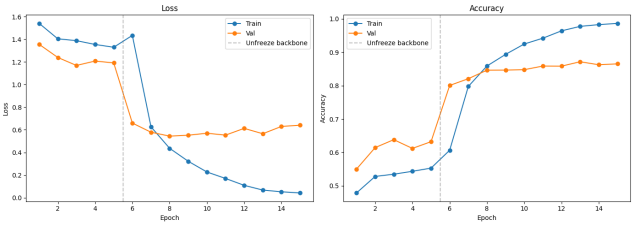



Figure 2: Training Curves for AlexNet

The progression of our training strategy is clearly reflected in the model's learning curves. During the initial phase (Epochs 1–5), where only the randomly initialized classifier head is trained, performance gains are steady but modest. This

phase successfully allows the head to calibrate without backpropagating large, disruptive gradients into the pre-trained convolutional backbone.

Upon unfreezing the backbone (marked by the dashed line), we observe a sharp, immediate acceleration in learning. The model rapidly adapts its deeper feature representations to the target audio domain, resulting in a steep decline in training loss and a corresponding surge in accuracy. However, as full fine-tuning progresses past epoch 8, the validation loss begins to gradually increase even as training loss continues to approach zero. This divergence, coupled with the plateauing of validation accuracy, indicates the onset of overfitting. These dynamics strongly validate our use of early stopping based on validation metrics, ensuring we capture the optimal model weights before the network begins to memorize the training data at the expense of generalization.

#### 2.1.2 Hyperparameter Tuning

To rigorously explore the hyperparameter space more efficiently, we utilized **Optuna**, an automatic hyperparameter optimization framework that leverages Bayesian optimization algorithms to sequentially construct a probabilistic model of the objective function. This allows it to dynamically focus its search on the most promising regions of the hyperparameter space.

Our Optuna search space was defined as follows:

- **Optimizer:** $\{\text{AdamW}, \text{SGD}\}$ (categorical)
- **Weight Decay:** $[10^{-6}, 10^{-2}]$ (log-uniform)
- **Backbone LR:** $[10^{-6}, 10^{-4}]$ (log-uniform)
- **Head LR:** $[10^{-5}, 10^{-3}]$ (log-uniform)

and converged on the following configuration:

| Optimizer | Weight Decay | Backbone LR | Head LR |
|---|---|---|---|
| AdamW | $6.41 \times 10^{-4}$ | $4.30 \times 10^{-5}$ | $1.77 \times 10^{-4}$ |

### 2.2 ResNet18

Next, we fine-tuned a **ResNet18**. This requires converting each 10-second audio chunk into an image-like input: we compute a 128-bin mel-spectrogram (2048-sample FFT window, 512-sample hop), convert to dB, min-max normalise to $[0, 1]$, and bilinearly resize to $224 \times 224$. The single-channel spectrogram is then replicated across 3 channels to match ResNet's RGB input. We replace the final 1000-class ImageNet head with a Dropout + Linear($512 \to 10$) head trained from scratch.

#### 2.2.1 Training and Hyperparameters

We use a two-phase transfer-learning schedule, like in the AlexNet section to safely adapt the ImageNet-pretrained weights to our bird spectrograms.

To see whether this could be improved, we ran a random search over 3 hyperparameters with 15 trials (3 Phase 1 epochs + 5 Phase 2 epochs per trial, ranking by peak validation accuracy). Random search was preferred over grid search because with 3 hyperparameters a modest grid is expensive. The search spanned:

- Batch size: $\{16, 32, 64\}$
- Learning rate: $[10^{-4}, 10^{-2}]$ (log-uniform)
- Dropout: $[0.1, 0.6]$ (uniform)

The best configuration found was:

| Batch size | LR | Dropout |
|---|---|---|
| 32 | $2.94 \times 10^{-4}$ | 0.53 |

Retraining with this configuration (and early stopping with patience 3) gave 0.8820 test chunk accuracy, below the 0.9002 baseline. This suggests the original defaults were already near-optimal for this task. We therefore kept the baseline weights as the final ResNet18 model.

### 2.3 Sound Event Detection (SED)

While standard image classification models (like a basic ResNet) treat spectrograms as static images, bird calls are inherently temporal events that often occupy only a fraction of a 10-second recording. To address this, we implemented a SED architecture using ResNet34 backbone equipped with a custom temporal attention head, resulting in approximately 21.6M trainable parameters.

#### 2.3.1 Audio Preprocessing

To capture the fine details of high-frequency bird sounds, we utilized an FFT window size of 2048. Since the dataset's recordings come from a wide variety of environments and equipment, we also built a custom per-sample normalization layer (`NormalizeMelSpec`). This layer standardizes the mean and variance of each individual spectrogram and scales it to a [0, 1] range. As a result, the model always receives consistent, high-contrast inputs, no matter how quiet or loud the original audio was. Then, once the audio is processed, we pass these normalized spectrograms through a pre-trained ResNet34 backbone. We chose this deeper 34-layer architecture because it provides a

rich enough feature hierarchy to distinguish between species that have very similar, overlapping spectral patterns. Finally, instead of treating the entire 10-second clip equally, our SED head uses an Attention Pooling mechanism to focus only on the relevant sounds:

1. GeMFreq Pooling: We start with Generalized Mean Pooling across the frequency dimension, which helps isolate the specific frequency bands where the birds are actually singing.

2. Temporal Attention Map: Next, the sequence of time frames goes through a MLP and 1D convolutions to create a temporal attention mask. This step basically figures out which moments in time are important, assigning a specific weight to each frame.

3. Clip-Level Aggregation: We calculate our prediction using a weighted sum of those frame-level logits. By assigning high weights (close to 1.0) to frames with bird calls and low weights (near 0.0) to silence, the model effectively learns to tune out the background noise.

#### 2.3.2 Optimization and Loss

Since real-world audio often contains multiple overlapping bird calls, we framed this as a multi-label problem. We optimized the network using `BCEWithLogitsLoss` so that the model evaluates each of the 10 target classes independently. The model was trained for 10 epochs using a Cosine Annealing learning rate scheduler alongside a validation-loss-based checkpointing system. This training strategy ensured we captured the optimal, most generalized version of our weights prior to the onset of overfitting. The final optimization hyperparameters are summarized in the table below.

| Optimizer | Weight Decay | LR | Batch Size | Epochs |
|---|---|---|---|---|
| AdamW | $1 \times 10^{-2}$ | $1 \times 10^{-3}$ | 16 | 10 |

### 2.4 Audio Spectrogram Transformer

The model we expected to perform best was the **Audio Spectrogram Transformer (AST)**, a Vision-Transformer variant pretrained on AudioSet (1.9M clips, 527 sound classes). The AST feature extractor takes a 10 s chunk at 16 kHz and computes a spectrogram with 128 frequency bins. That image is cut into $16 \times 16$ patches with stride 10 on both axes (i.e. overlapping patches), giving a 1,212-token grid that is linearly embedded and, together with a learned **[CLS] token** prepended to the sequence, passed to a 12-layer, 12-head transformer encoder ($\approx$ 86 M parameters). The [CLS] token carries no audio content of its own; its job is to aggregate information from all 1,212 patch tokens via self-attention as they flow through the 12 blocks, and only its final representation is fed to the classifier head so after fine-tuning, everything the model has decided about the clip lives in that single token. Figure 3 shows exactly what the model sees: the raw mel-spectrogram on the left, and the patchified token grid that actually enters the transformer on the right.

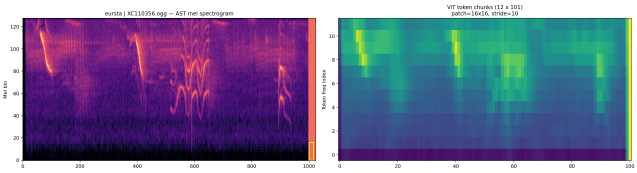


Figure 3: Left: $128 \times 1024$ log-mel spectrogram of a 10 s chunk. Right: the same input after $16 \times 16$ patchification with stride 10, i.e. the $12 \times 101$ token grid fed to the transformer.

#### 2.4.1 Effects of transfer learning

AudioSet pretraining is the whole point of choosing AST. In classical audio pipelines, practitioners *hand-crafted* features on top of the spectrogram: spectral contrast, chroma, onset envelopes because the raw mel image is too high-dimensional to classify directly. During AudioSet pretraining, AST's self-attention heads had to learn to discriminate 527 heterogeneous sound classes (music, speech, animal calls, machinery, environmental noise). In the process they effectively *learned* a general-purpose bank of acoustic features, replacing those hand-crafted descriptors with data-driven ones. Fine-tuning on our 10-species subset therefore reuses this learned feature bank and only has to learn how to re-weight it via the classifier head plus a small update to the attention layers which is a far lighter task than learning features from scratch on 4,000 recordings.

#### 2.4.2 Hyperparameter Search

A single 5-epoch fine-tune on the full training set takes $\approx$ 65 min on an L40S, so an exhaustive sweep is not affordable. We deliberately restricted the search to the two hyperparameters we expected to actually matter given the rest of the recipe is a standard ViT fine-tuning recipe: the **learning rate** (how aggressively the pretrained weights are perturbed, which is the usual failure mode when fine-tuning large pretrained transformers) and the **weight decay** (the main regulariser against overfitting a 4,000-recording dataset). Batch size (16), warmup (50 steps), optimiser (AdamW) and loss

(cross-entropy) were fixed at values that are known to work for ViT-scale fine-tunes.

We sampled 16 log-uniform configurations — LR $\in [10^{-6}, 10^{-3}]$, WD $\in [10^{-5}, 10^{-1}]$ — and trained each on a stratified 25 % subsample of the training set. The 16 jobs were dispatched concurrently across 10 L40S GPUs; the sweep that would have taken $\sim 5$ h serially finished in well under an hour, which is what made the search tractable.

The sweep turned out to be *not very informative*. Validation accuracy ranged only from 0.84 to 0.90 across all 16 configurations, and the top three spanned nearly two orders of magnitude in both LR ($1.3 \times 10^{-5}$ to $2.1 \times 10^{-4}$) and WD ($5.7 \times 10^{-4}$ to $7.4 \times 10^{-2}$) This is consistent with the pretraining story above: once LR and WD are in a broadly reasonable range, a well-pretrained AST is robust to their exact values. We therefore retrained all three rather than trusting a single "winner" on the full training set for 5 epochs and reported the winner in the performance analysis.

# 3 Performance Analysis

## 3.1 Summary Results

| Config | Chunk Acc | Record Acc |
|---|---|---|
| AlexNet | 0.8622 | 0.8780 |
| ResNet | 0.9002 | 0.9240 |
| SED | 0.8056 | 0.8900 |
| AST | **0.9422** | **0.9500** |

We notice that our models always perform better on record accuracy than chunk accuracy which makes sense since it allows us to bypass some misclassifications when we only misclassified one of multiple chunks in a recording. Anyways, as mentioned previously, the chunk score does not make sense for final evaluation since it does not reflect the real world applications. When looking at the record accuracy, we see that the best model turns out to be the AST as expected. It reaches 95% record accuracy on the test data, closely followed by our ResNet at 92.4%.

## 3.2 Per-Class Performance

We can also observe different per-class (i.e per-bird species) performance of each model. Because of the balance of the classes, we can assume that any discrepancy in class accuracy is caused by certain birds having a more complicated song to recognize compared to others which might be simpler. This is perfectly exemplified by the fact that all our models predict the common raven (comrav in the data) with 100% accuracy since its cry is deep, loud and distinguishable. On the other hand, our models consistently underperform on the European starling (eursta in the data), showing accuracies between 68-75% depending on the model. This is likely because European starlings are well known for their variable song, which often attempt to mimic other birds, and therefore might confuse our models.

# 4 Limitations

1. **Dataset:** 5,000 recordings (500/species) is modest by deep learning standards. More data would likely improve performance. However, such data is not easy to come by because Bird call labeling needs to be done by ornithologists or serious bird watchers.

2. **Recording length heterogeneity:** The dataset contains clips ranging from 8-30 seconds. Longer clips are chunked, potentially losing temporal context at chunk boundaries.

3. **Acoustic diversity:** Recordings from Xeno-Canto, introduce author, equipment, and recording-condition variation which may degrade performance.

4. **Hyperparameter search:** Because of the complexity of the models and the computation power limitations, we were not able to search the full hyperparameter space for each of our models, meaning that further optimizations might be possible.

# 5 Conclusion

We were able to successfully recognize a subset of 10 bird-species from BirdCLEF 2021, achieving 95% recording-level test accuracy with our AST model. The pretrained AudioSet backbone combined with transfer learning and complete hyperparameter exploration allows it to generalize well despite modest dataset size, underscoring the value of large-scale pretraining for audio tasks. Our other models achieve high accuracies as well, such as the ResNet achieving 92.4%. This shows that machine learning techniques are able to distinguish complex audios like bird songs, however it necessitates expensive models, even when dealing with few classes (10 instead of the full 397 in BirdCLEF 2021).